Reading 7.2: GAN: Loss Function and Optimisation Problem

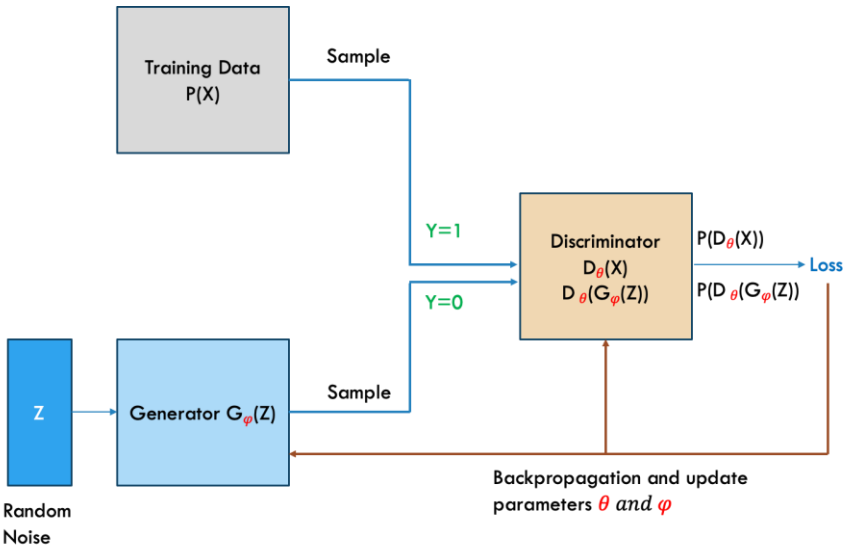
Every Machine Learning problem can be viewed as an optimisation problem where the goal is to minimise a defined loss function. Throughout this process, it's crucial to monitor and mitigate overfitting to ensure the model generalises well to unseen data.

We start with a dataset of images X that are distributed according to an unknown probability distribution P(X). This dataset represents our training data.

The generator G, parametrised by φ, takes a random noise vector Z as input, where Z is sampled from a simple distribution P(Z), Gaussian or uniform. G subsequently outputs a generated image. This generated image is then passed to D, parametrised by θ.

D assesses the generated image and calculates two probabilities: the likelihood that the image originates from the training dataset and the likelihood that it was generated by G, hence being real or fake. D tries to distinguish between real and generated (fake) images, while G tries to fool D. G's end game is to generate images so realistic that D cannot differentiate between real and generated images. This can be expressed by following equation:

$$\text{End Game for } G_{\varphi}(Z): \; P(D_{\theta}(X)) = P(D_{\theta}(G_{\varphi}(Z)) = 50\%$$



During training, a loss function is computed to measure the performance of both models. This loss is then backpropagated to update the parameters θ and φ through iterative optimisation steps, refining both networks over time.

Let's formalise the optimisation problem by introducing a loss function.

D's role is to act as a binary classifier, and calculates two probabilities: the probability that an image comes from the training data (real) and the probability that the image is generated by G (fake). Going forward we will not explicitly write P($D_\theta(X)$) and P($D_\theta(G_\varphi$(Z)). We will consider $D_\theta(X)$ and $D_\theta(G_\varphi$(Z)) as probabilities. We can rely on the binary cross-entropy expression as the loss function.

Real Image

$$L(y_r, \hat{y}) = y log y_r + (1-y)\log(1-y_r) \quad (1)$$

Reconstructed Image
D(X) or D(G(Z)

D wants to maximise P[(X)] and minimise P[(Z)].

- P(X)→1 (maximise)
- P(Z)→ 0 (minimise) ↔ (1-P(Z))→1 (maximise)

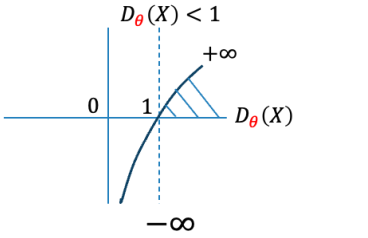
Both probabilities are now to be maximised!

Let's continue exploring the loss function by defining the optimisation problem for G. We've already defined the objective for D, which aims to maximise its ability to distinguish real images (Y=1) from fake ones (Y=0).

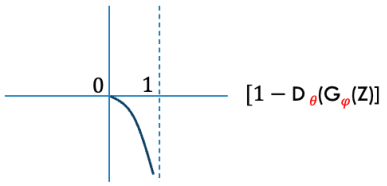
In the scenario Y=1, the image is coming from training data, D aims to output a probability that is close to 1. In the scenario Y=0, the image is generated by G and D aims to output a probability that is close to 0.

$D_\theta(X)$ must be ≤ 1 as probability

Y=1 → (1) $L(y_r, y) = log D_\theta(X)$



Y=0 → (1) $L(y_r, y) = \log[1 - D_\theta(G_\varphi(Z)]$



For D our optimisation problem is:

$$Discriminator: \max_\theta D \{log D_\theta(X) + log[1 - D_\theta(G_\varphi(Z)\}$$

Now, let's focus on G. G's goal is to *fool* the discriminator. Fooling the discriminator means maximising the probability that D classifies fake images as real: P(Z)→1 or minimise (1-P(Z)) →0

To maximise synergies for the optimisation expression we will keep $logD_\theta(X)$ in the optimisation expression for G. As G does not depend on θ this term will disappear when taking the gradient with respect to φ .When taking the derivative with respect to φ, this terms will disappear anyway.

$$Generator: \quad \min_{\varphi} G \{logD_\theta(X) + \log[1 - D_\theta(G_\varphi(Z)\}$$

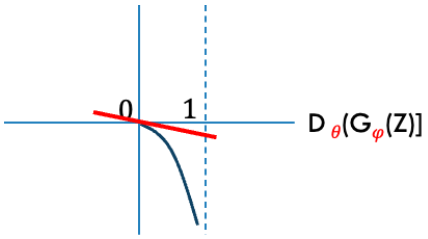
The loss function for a GAN can be written as follows:

$$\min_{\varphi} G \max_{\theta} D \{E_{X\sim P(X)}[logD_\theta(X)] + E_{Z\sim P(Z)}[\log[1 - D_\theta(G_\varphi(Z)]\}$$

Note that we added Expected Values (E) in the formula, as we average out over all training samples and their probability of occurrence. We clearly see that the optimisation problem is in fact a minmax problem, known from Game Theory. A minimax problem always had a solution, called the Nash Equilibrium. A Nash equilibrium is a stable state where no player can improve their outcome by unilaterally changing their strategy.

Objective Function for GANs

Minimising the objective function for G is an issue as we start training G.



Initially, when D is highly confident and assigns a probability near zero to fake images, the gradient will be flat potentially making our optimisation problem unstable.

To resolve this, instead of minimising $log[1 - D_\theta(G_\varphi(\mathrm{Z})]$ we modify G's objective to maximising $log[D_\theta(G_\varphi(\mathrm{Z})]$. This provides us higher gradient signals for bad samples. We maximise the likelihood of D being wrong instead of minimising the likelihood of D being correct. Our optimisation problem becomes:

$$\max_{\varphi} G \max_{\theta} D \{E_{X\sim P(X)}[logD_\theta(X)] + E_{Z\sim P(Z)}[\log \mathrm{D}_\theta(\mathrm{G}_\varphi(\mathrm{Z})]\}$$

This small adjustment provides a better gradient to work with, especially during the early stages of training, where improvements are most critical.